Project 1: Dynamic Programming

Shambhavi Singh

I. Introduction

This project studies autonomous navigation in a Door-Key grid environment, where an agent must reach a goal location while interacting with obstacles. The environment consists of a grid with walls, a locked door, a key, and a goal. The agent (represented as a red triangle indicating its orientation) must plan a sequence of actions to reach the goal (green cell). If the door blocks the path, the agent must first navigate to the key, pick it up, and unlock the door before proceeding. The environment is deterministic, meaning state transitions are fully determined by the current state and action.

This problem models key challenges in robotics, including task sequencing, obstacle avoidance, and interaction with the environment. Similar formulations arise in autonomous driving (navigating roads with constraints), warehouse robotics (retrieving items and navigating aisles), and service robots (cleaning or stocking in structured spaces).

We formulate the problem as a finite-horizon Markov Decision Process and solve it using dynamic programming to compute an optimal control policy. For Part A, the policy is computed separately for 7 known environments. For Part B, a single policy is computed that generalizes across 36 random environments by incorporating variations such as key location, goal location, and door states directly into the state space. The objective is to minimize the total action cost, which corresponds to finding the shortest feasible sequence of actions that reaches the goal while respecting constraints such as picking up the key and unlocking the door when required.

II. Problem Statement

We formulate the Door-Key navigation task as a finite-horizon deterministic Markov Decision Process (MDP)

$$\mathcal{M} = (\mathcal{X}, \mathcal{U}, f, x_0, T, \ell, q).$$

The goal is to define the state, action space, deterministic transition model, planning horizon, stage cost, and terminal cost for both the known and random map settings.

A. Part A: Known Maps

For each known environment, we compute a separate policy. The state is defined as

$$x = (p_x, p_y, d, k, o),$$

where (p_x, p_y) is the agent position, $d \in \{(1,0),(0,1),(-1,0),(0,-1)\}$ is the agent direction, $k \in \{0,1\}$ indicates whether the agent has the key, and $o \in \{0,1\}$ indicates whether the door is open. Therefore, the state space is

$$\mathcal{X} = \{0,\dots,W-1\} \times \{0,\dots,H-1\} \times \mathcal{D} \times \{0,1\} \times \{0,1\}.$$

The control space is

$$\mathcal{U} = \{\text{MF}, \text{TL}, \text{TR}, \text{PK}, \text{UD}\},$$

where MF moves forward, TL turns left, TR turns right, PK picks up the key, and UD unlocks the door.

The transition model is deterministic:

$$x_{t+1} = f(x_t, u_t).$$

The MF action moves the agent one cell forward if the next cell is in bounds and is not blocked by a wall or closed door. TL and TR update only the agent direction. PK sets $k = 1$ if the key is directly in front of the agent. UD sets $o = 1$ if the door is directly in front of the agent and $k = 1$. Invalid actions are excluded from the feasible action set.

The initial state is

$$x_0 = (p_{x,0}, p_{y,0}, d_0, 0, o_0),$$

where $(p_{x,0}, p_{y,0})$, d_0, and o_0 are read from the environment.

We choose the planning horizon as

$$T = |\mathcal{X}| - 1.$$

Since the problem is deterministic and all stage costs are positive, an optimal trajectory does not need to revisit the same state. Thus, $|\mathcal{X}| - 1$ is a conservative upper bound on the number of transitions.

The stage cost is

$$\ell(u) = \begin{cases} 1, & u \in \{\text{TL}, \text{TR}\}, \\ 2, & u = \text{PK}, \\ 3, & u = \text{MF}, \\ 5, & u = \text{UD}. \end{cases}$$

The terminal cost is

$$q(x) = \begin{cases} 0, & (p_x, p_y) = p_{\text{goal}}, \\ +\infty, & \text{otherwise}. \end{cases}$$

The finite-horizon planning problem is to find a policy that minimizes the total cost:

$$\pi^* = \arg\min_{\pi} \left[\sum_{t=0}^{T-1} \ell(u_t) + q(x_T) \right]$$

subject to

$$\begin{aligned} x_{t+1} &= f(x_t, u_t), \qquad t = 0, \dots, T-1, \\ x_t &\in \mathcal{X}, \qquad u_t \in \mathcal{U}(x_t), \\ u_t &= \pi_t(x_t), \qquad x_0 = x_{\text{init}}. \end{aligned}$$

Here, $\mathcal{U}(x_t)$ is the set of valid controls from state x_t. This means actions that would move into a wall, move through a closed door, pick up a nonexistent key, or unlock a door without the key are not feasible controls.

B. Part B: Random Maps

For the random maps, we compute one shared policy that works for all 36 environments. To do this, the map variation is included directly in the state. The state is

$$x = (p_x, p_y, d, k, o_1, o_2, p_{\text{key}}, p_{\text{goal}}),$$

where $o_1, o_2 \in \{0, 1\}$ represent the open/closed status of the two doors at $(5, 3)$ and $(5, 7)$. The possible key positions are

$$p_{\text{key}} \in \{(2, 2), (2, 3), (1, 6)\},$$

and the possible goal positions are

$$p_{\text{goal}} \in \{(6, 1), (7, 3), (6, 6)\}.$$

Therefore,

$$|\mathcal{X}| = 10 \cdot 10 \cdot 4 \cdot 2 \cdot 2 \cdot 2 \cdot 3 \cdot 3 = 28800.$$

The control space is the same as in Part A:

$$\mathcal{U} = \{\text{MF}, \text{TL}, \text{TR}, \text{PK}, \text{UD}\}.$$

The transition model is again deterministic:

$$x_{t+1} = f(x_t, u_t).$$

The MF action is invalid if the agent attempts to move out of bounds, into a wall, or through a closed door. The two door cells $(5, 3)$ and $(5, 7)$ are passable only when their corresponding door state is open. The PK action sets $k = 1$ when the key is in front of the agent. The UD action opens the corresponding door when the agent has the key and is facing that door.

For each random environment, the initial agent position and direction are fixed:

$$(p_{x,0}, p_{y,0}) = (4, 8), \qquad d_0 = (0, -1).$$

Thus,

$$x_0 = (4, 8, (0, -1), 0, o_1, o_2, p_{\text{key}}, p_{\text{goal}}),$$

where $o_1, o_2, p_{\text{key}}, p_{\text{goal}}$ depend on the specific random map.

We choose the planning horizon as

$$T = |\mathcal{X}| - 1 = 28799.$$

This gives a conservative upper bound on the number of transitions because the optimal path should not need to revisit the same state.

The stage cost is the same as in Part A:

$$\ell(u) = \begin{cases} 1, & u \in \{\text{TL}, \text{TR}\}, \\ 2, & u = \text{PK}, \\ 3, & u = \text{MF}, \\ 5, & u = \text{UD}. \end{cases}$$

The terminal cost is

$$q(x) = \begin{cases} 0, & (p_x, p_y) = p_{\text{goal}}, \\ +\infty, & \text{otherwise}. \end{cases}$$

The finite-horizon planning problem is to find a policy that minimizes the total cost:

$$\pi^* = \arg\min_{\pi} \left[\sum_{t=0}^{T-1} \ell(u_t) + q(x_T) \right]$$

subject to

$$x_{t+1} = f(x_t, u_t), \qquad t = 0, \ldots, T-1,$$
$$x_t \in \mathcal{X}, \qquad u_t \in \mathcal{U}(x_t),$$
$$u_t = \pi_t(x_t), \qquad x_0 = x_{\text{init}}.$$

Here, $\mathcal{U}(x_t)$ is the set of valid controls from state x_t. This means actions that would move into a wall, move through a closed door, pick up a nonexistent key, or unlock a door without the key are not feasible controls.

III. Technical Approach

To solve the Door-Key environments, I implemented finite-horizon dynamic programming over the full discrete state space. Since the environment is deterministic and the number of possible states is finite, dynamic programming is a direct way to compute an optimal control policy. The method computes the minimum cost-to-go from every state and stores the best action to take from that state.

A. State Space Construction

The first step is to explicitly construct the state space. For Part A, the function `build_state_space()` enumerates all possible combinations of agent position, direction, key possession, and door status:

$$x = (p_x, p_y, d, k, o).$$

This is done for each known environment separately because each known map can have different dimensions, walls, key position, door position, and goal position.

For Part B, the function `build_state_space_B()` constructs a larger state space:

$$x = (p_x, p_y, d, k, o_1, o_2, p_{\text{key}}, p_{\text{goal}}).$$

This includes the two door states, the key position, and the goal position. I included p_{key} and p_{goal} directly in the state because the random environments can have different key and goal locations. With this formulation, one shared policy can be computed and then applied to all 36 random maps.

B. Valid Action Checking

Before applying the transition function, the implementation checks whether each action is valid. This is done using `valid_action()` for Part A and `valid_action_B()` for Part B. This choice is important because invalid actions should not be included in the minimization.

For example, the move-forward action is invalid if the next cell is out of bounds, a wall, or a closed door. The pickup-key action is valid only when the key is directly in front of the agent and the agent does not already have the key. The unlock-door action is valid only when the agent has the key

and is facing a closed door. Turn-left and turn-right are always valid because they only change the agent's orientation.

By excluding invalid actions, the dynamic programming update only considers feasible controls:

$$u \in \mathcal{U}(x).$$

C. Transition Model

The transition model is deterministic and is implemented using `pf()` for Part A and `pf_B()` for Part B. Given a current state and a valid action, the next state is uniquely determined:

$$x_{t+1} = f(x_t, u_t).$$

For move-forward, the agent position is updated by adding the direction vector to the current position. For turn-left and turn-right, only the direction component changes. For pickup-key, the key possession variable is updated from 0 to 1. For unlock-door, the appropriate door-open variable is updated from 0 to 1.

This deterministic transition model allows the dynamic programming update to use a single next state $f(x, u)$ rather than taking an expectation over possible next states.

D. Cost Design

The stage cost function is implemented in `l(action)`. I used the action costs specified in the project:

$$\ell(u) = \begin{cases} 1, & u \in \{\text{TL}, \text{TR}\}, \\ 2, & u = \text{PK}, \\ 3, & u = \text{MF}, \\ 5, & u = \text{UD}. \end{cases}$$

These costs make the planner minimize total action cost rather than only minimizing the number of actions. Therefore, the optimal path is the lowest-cost feasible path to the goal.

The terminal cost is implemented using `q()` for Part A and `q_B()` for Part B:

$$q(x) = \begin{cases} 0, & (p_x, p_y) = p_{\text{goal}}, \\ +\infty, & \text{otherwise}. \end{cases}$$

This forces the final state to be at the goal. If the final state is not the goal, the trajectory receives infinite terminal cost.

E. Planning Horizon

The planning horizon is chosen as

$$T = |\mathcal{X}| - 1.$$

This is a conservative upper bound on the number of transitions needed. Since the environment is deterministic and all stage costs are positive, an optimal trajectory should not need to revisit the same state. Revisiting a state would create a cycle with positive cost, which can be removed to obtain a lower-cost or equal-feasibility path. Therefore, the longest simple path can visit at most every state once, giving at most $|\mathcal{X}| - 1$ transitions.

For Part A, this horizon is computed separately for each known map using the size of that map's state space. For Part B, the full state space has

$$|\mathcal{X}| = 10 \cdot 10 \cdot 4 \cdot 2 \cdot 2 \cdot 2 \cdot 3 \cdot 3 = 28800,$$

so the horizon is

$$T = 28799.$$

F. Backward Dynamic Programming

I solve the finite-horizon MDP using backward dynamic programming. The value function $V_t(x)$ stores the minimum cost-to-go from state x at time t. The policy $\pi_t(x)$ stores the action that achieves this minimum.

The terminal value is initialized as

$$V_T(x) = q(x).$$

Then, for each time step $t = T - 1, \ldots, 0$, and for each state $x \in \mathcal{X}$, the algorithm evaluates every valid action:

$$V_t(x) = \min_{u \in \mathcal{U}(x)} \left[\ell(u) + V_{t+1}(f(x, u))\right].$$

The action that achieves this minimum is stored as the optimal policy:

$$\pi_t^*(x) = \arg\min_{u \in \mathcal{U}(x)} \left[\ell(u) + V_{t+1}(f(x, u))\right].$$

In the code, this is implemented by creating a dictionary `Q` for each state. For every valid action, the next state is computed using the transition function, and the cost

$$\ell(u) + V_{t+1}(f(x, u))$$

is stored. The action with the smallest value in `Q` is selected using `min(Q, key=Q.get)` and stored in the policy dictionary.

G. Convergence Check

The implementation also includes a convergence check. After computing $V_t(x)$, the code compares it to $V_{t+1}(x)$. If the value function does not change for any state, the algorithm has converged and can stop early:

$$V_t(x) = V_{t+1}(x) \quad \forall x \in \mathcal{X}.$$

This avoids unnecessary backward iterations once the optimal cost-to-go has stabilized.

In the implementation, this check must be done over all states, not only one state. Therefore, after each state update, the code checks whether $V_t(x) \neq V_{t+1}(x)$. If any state changes, the algorithm continues. If no states change, the policy and value function are returned.

H. Algorithm

Input: state space $\mathcal{X}$, control space $\mathcal{U}$, transition function f, horizon T, stage cost ℓ, terminal cost q
Output: policy π and value function V
for each state $x \in \mathcal{X}$ **do**
  $V_T(x) \leftarrow q(x)$
end for
for $t = T-1, T-2, \ldots, 0$ **do**
  for each state $x \in \mathcal{X}$ **do**
    if $q(x) = 0$ **then**
      $V_t(x) \leftarrow 0$
      $\pi_t(x) \leftarrow$ None
    else
      $Q \leftarrow \emptyset$
      for each action $u \in \mathcal{U}$ **do**
        if u is valid from state x **then**
          $x' \leftarrow f(x, u)$
          $Q(u) \leftarrow \ell(u) + V_{t+1}(x')$
        end if
      end for
      $\pi_t(x) \leftarrow \arg\min_u Q(u)$
      $V_t(x) \leftarrow \min_u Q(u)$
    end if
  end for
end for
return π, V

I. Part A Implementation

For Part A, I compute a separate policy for each of the seven known maps. This is necessary because each known map has its own layout and environment information, including wall locations, key position, door position, goal position, and map size.

For each map, the program:

1) Loads the environment using `load_env()`.
2) Extracts wall positions using `get_walls()`.
3) Builds the state space using `build_state_space()`.
4) Reads the initial agent position, direction, key status, and door status.
5) Runs `dpa()` to compute the optimal policy and value function.
6) Extracts the action sequence using `get_action_sequence()`.
7) Saves the resulting path as a GIF using `draw_gif_from_seq()`.

J. Part B Implementation

For Part B, I compute one shared policy for all 36 random maps. Instead of solving a separate MDP for each random map, I include the varying key and goal locations in the state. This makes the policy depend on p_{key} and p_{goal}, so the same policy can handle different random maps.

The program first builds the full Part B state space and runs `dpa_B()` once. Then, for each random environment, it reads the actual key position, goal position, and door states from the map. These values are inserted into the initial state:

$$x_0 = (4, 8, (0, -1), 0, o_1, o_2, p_{\text{key}}, p_{\text{goal}}).$$

The program then follows the shared policy from this initial state to extract the action sequence for that specific environment. For Part B, the program:

1) Builds the full Part B state space using `build_state_space_B(10, 10)`.
2) Sets the planning horizon to $T = |\mathcal{X}| - 1$.
3) Runs `dpa_B()` once to compute a shared optimal policy and value function.
4) Loops through the 36 random environments.
5) Loads each random environment using `load_env()`.
6) Reads the two door states, key position, and goal position from the environment.
7) Constructs the initial state
$$x_0 = (4, 8, (0, -1), 0, o_1, o_2, p_{\text{key}}, p_{\text{goal}}).$$
8) Extracts the action sequence using `get_action_sequence_B()`.
9) Saves the resulting path as a GIF using `draw_gif_from_seq()`.

K. Action Sequence Extraction

After the policy is computed, the final path is obtained by simulating the policy from the initial state. Starting from x_0, the program repeatedly looks up the stored optimal action, appends that action to the sequence, and applies the transition function to reach the next state. This continues until the terminal condition is reached:

$$(p_x, p_y) = p_{\text{goal}}.$$

The resulting sequence is the planned solution for that map.

Finally, the action sequence is passed to `draw_gif_from_seq()`, which generates a GIF visualization of the agent moving through the environment.

IV. Results

The dynamic programming planner successfully generated feasible paths for all seven known maps and all thirty-six random maps. The resulting action sequences and total costs are shown below.

A. Part A Results

For Part A, (Fig. 1) the dynamic programming planner successfully computed feasible action sequences for all seven known Door-Key maps. In each case, the generated action sequence reached the goal while respecting the environment constraints. The agent did not move through walls or closed doors, and when the door was required, the policy first picked up the key and then unlocked the door before moving through it.

The results show that the planner correctly adapts to different map layouts. For the direct maps, such as `6x6-direct.env` and `8x8-direct.env`, the optimal

paths are much shorter because the agent can reach the goal without needing to pick up the key or unlock the door. These maps have lower total costs of 13 and 17, respectively. In contrast, the normal maps require the agent to collect the key and unlock the door, which leads to longer action sequences and higher costs. For example, `8x8-normal.env` has the highest Part A cost of 54 because the agent must travel farther while also performing the pickup and unlock actions.

The shortcut maps also worked as expected. In `6x6-shortcut.env` and `8x8-shortcut.env`, the planner uses the key and door when doing so gives a lower-cost path to the goal. This confirms that the policy is not simply minimizing the number of moves, but is minimizing the total action cost based on the specified cost function.

Overall, Part A worked well for all seven known maps. The main limitation is that a separate dynamic programming policy must be computed for each known environment. This is because each map has different walls, key location, door location, goal location, and map size. Therefore, the Part A implementation is effective for fixed known maps, but it does not directly generalize across multiple map layouts without recomputing the policy.

The generated trajectories show that the dynamic programming policy successfully moves the agent from the initial state to the goal while respecting the Door-Key constraints. In the visualizations, the red triangle represents the agent, the yellow key represents the key location, the yellow outlined cell represents the door, and the green cell represents the goal. Across the shown examples, the agent follows a feasible sequence of actions and reaches the goal without crossing walls or passing through a locked door.

B. 6x6 Result Discussion

The 6x6 results can be grouped into three map types: direct, normal, and shortcut. These maps have the same grid size but different layouts, so they show how the dynamic programming policy changes depending on whether the key-door mechanism is necessary.

1) Direct Map: In the `6x6-direct.env` case (Fig. 2), the agent reaches the goal directly without needing to pick up the key or unlock the door. The action sequence is

MF, MF, TR, MF, MF

with total cost 13. This worked as expected because the goal is reachable through an open path. The policy correctly avoids the key and door actions because using `PK` or `UD` would only add unnecessary cost.

2) Normal Map: In the `6x6-normal.env` case (Fig. 3), the agent must use the key and door to reach the goal. The trajectory first moves toward the key, performs `PK`, navigates toward the door, performs `UD`, and then moves through the door to the goal. The total cost is 33. This confirms that the state representation correctly tracks key possession and door status. The agent does not attempt to move through the door before unlocking it.

3) Shortcut Map: In the `6x6-shortcut.env` case (Fig. 4), the key and door provide a shorter route to the goal. The action sequence is

PK, TL, TL, UD, MF, MF

with total cost 15. The policy immediately uses the key-door mechanism because it gives a lower-cost route than moving around the map. This shows that the planner is minimizing total action cost, not just avoiding door interactions.

Overall, the 6x6 results show that the dynamic programming planner adapts correctly to the map structure. In the direct map, it ignores the key and door because they are unnecessary. In the normal map, it follows the required key-door sequence. In the shortcut map, it uses the key and door because they reduce the path cost.

C. Part B Results

For Part B (Fig. 5), the planner was evaluated on 36 random 10x10 Door-Key environments. Unlike Part A, these maps are not solved with separate policies. Instead, one shared policy is computed over an expanded state space that includes the key location, goal location, and the open/closed status of both doors. The results show that this shared policy successfully generates feasible action sequences for all 36 random environments.

The costs vary across the random maps because the key and goal positions change. Some maps have low costs, such as `DoorKey-10x10-9.env`, `DoorKey-10x10-11.env`, `DoorKey-10x10-21.env`, `DoorKey-10x10-23.env`, `DoorKey-10x10-33.env`, and `DoorKey-10x10-35.env`, each with cost 14. In these cases, the agent can reach the goal through a short path without needing to pick up the key or unlock a door. The action sequences only contain movement and turning actions, showing that the planner correctly avoids unnecessary key-door interactions when they are not needed.

Other maps require the key-door sequence. For example, maps such as `DoorKey-10x10-4.env`, `DoorKey-10x10-8.env`, `DoorKey-10x10-12.env`, `DoorKey-10x10-16.env`, `DoorKey-10x10-20.env`, `DoorKey-10x10-24.env`, `DoorKey-10x10-28.env`, `DoorKey-10x10-32.env`, and `DoorKey-10x10-36.env` include both `PK` and `UD` in their action sequences. This means the policy first moves to the key, picks it up, navigates to the correct door, unlocks it, and then proceeds toward the goal. These maps have higher costs because they require extra travel and the pickup/unlock actions.

The highest costs occur in maps where the agent must take a longer route involving the key and door. For example, `DoorKey-10x10-12.env` has cost 53, while `DoorKey-10x10-4.env` and `DoorKey-10x10-28.env` both have cost 50. These higher values are expected because the agent must move farther through the grid and perform additional actions before reaching the goal. In contrast, maps with cost 25–32

typically involve mostly forward movements and turns, without requiring the full key-door interaction.

Overall, the Part B results show that the expanded state representation works as intended. By including p_{key}, p_{goal}, o_1, and o_2 in the state, the same dynamic programming policy can adapt to different random environments. The policy chooses shorter direct paths when possible and uses the key and door only when required or beneficial. The main limitation is computational cost: the Part B state space contains 28800 states, so computing the shared policy is much more expensive than solving the smaller known maps in Part A.

D. Part B Representative Map Discussion

For Part B, I selected `DoorKey-10x10-9.env`, `DoorKey-10x10-11.env`, and `DoorKey-10x10-12.env` as representative random maps. These maps are useful because they show how the shared policy changes depending on the initial door configuration.

1) `DoorKey-10x10-9.env`:

In `DoorKey-10x10-9.env`(Fig. 6), both doors are already open. The agent reaches the goal with the short action sequence

MF, TR, MF, MF, TL, MF

with total cost 14. Since both doors are open, the agent does not need to pick up the key or perform the unlock action. The policy correctly avoids unnecessary `PK` and `UD` actions and simply follows the lowest-cost route to the goal.

2) `DoorKey-10x10-11.env`:

In `DoorKey-10x10-11.env` (Fig. 7), one door is already open, specifically the door at $(5, 7)$. The action sequence is

MF, TR, MF, MF, TL, MF

with total cost 14. Similar to `DoorKey-10x10-9.env`, the agent can reach the goal without collecting the key or unlocking a door. This shows that the policy uses the open-door information in the state and does not force the key-door sequence when an open route is available.

3) `DoorKey-10x10-12.env`:

In `DoorKey-10x10-12.env` (Fig. 8), both doors are closed. The action sequence is

MF, MF, MF, MF, MF, MF, TL, MF, PK, TL,
MF, TL, MF, UD, MF, MF, TR, MF, MF, MF

with total cost 53. Unlike the previous two maps, the agent must pick up the key and unlock a door before reaching the goal. The higher cost is expected because the agent has to travel to the key, perform `PK`, move to the door, perform `UD`, and then continue to the goal.

Overall, these three maps show that the shared Part B policy adapts correctly to the door configuration. When doors are already open, as in `DoorKey-10x10-9.env` and `DoorKey-10x10-11.env`, the policy takes the direct route and avoids unnecessary key or unlock actions. When both doors are closed, as in `DoorKey-10x10-12.env`, the policy correctly follows the required key-door sequence before moving to the goal.

E. Performance Analysis

Overall, the dynamic programming algorithm worked correctly for both the known and random Door-Key environments. For every tested map, the planner produced a feasible action sequence that reached the goal while respecting the constraints of the environment. The agent avoided walls, did not pass through closed doors, picked up the key when needed, and unlocked the door only after obtaining the key.

For Part A, the algorithm performed well because each known map has a relatively small state space. Since the maps are fixed, a separate policy can be computed for each environment using its specific wall positions, key position, door position, and goal position. This allowed the planner to find low-cost paths for the direct, normal, and shortcut maps. In the direct maps, the policy avoided unnecessary key and door actions. In the normal maps, it correctly followed the required key-door sequence. In the shortcut maps, it used the key and door when they provided a lower-cost path.

For Part B, the shared-policy approach also worked. By including the key position, goal position, and door states in the state representation, the same policy could be applied to all 36 random maps. This was effective because the policy had enough information to choose different actions depending on the map configuration. For example, when doors were already open, the policy often moved directly to the goal. When both doors were closed and blocked the useful path, the policy picked up the key and unlocked the required door.

The main limitation of the approach is computational cost. Dynamic programming requires iterating over the full state space and evaluating valid actions for each state. This is manageable for the smaller known maps, but the Part B state space is much larger:

$$|\mathcal{X}| = 28800.$$

As a result, computing the shared policy for Part B takes significantly more time and memory than solving one known map in Part A.

Another limitation is that the method depends on a complete discrete state representation. If the map size increased or more objects were added, the state space would grow quickly. This makes full state-space dynamic programming less scalable for larger environments. However, for this project, the finite state spaces were small enough that the method could compute optimal policies and generate correct solution paths.

Map	Cost	Action Sequence
5x5-normal.env	28	TL, TL, MF, TR, PK, MF, TR, UD, MF, TL, MF, TR, MF
6x6-normal.env	33	MF, TL, MF, PK, TL, MF, TL, MF, TR, UD, MF, MF, TR, MF
6x6-direct.env	13	MF, MF, TR, MF, MF
6x6-shortcut.env	15	PK, TL, TL, UD, MF, MF
8x8-normal.env	54	MF, TR, MF, MF, MF, MF, PK, TL, TL, MF, MF, MF, TR, UD, MF, MF, MF, TR, MF, MF, MF
8x8-direct.env	17	MF, TL, MF, MF, MF, TL, MF
8x8-shortcut.env	19	TR, MF, TR, PK, TL, UD, MF, MF

Fig. 1. Action sequences and total costs for the Door-Key maps.

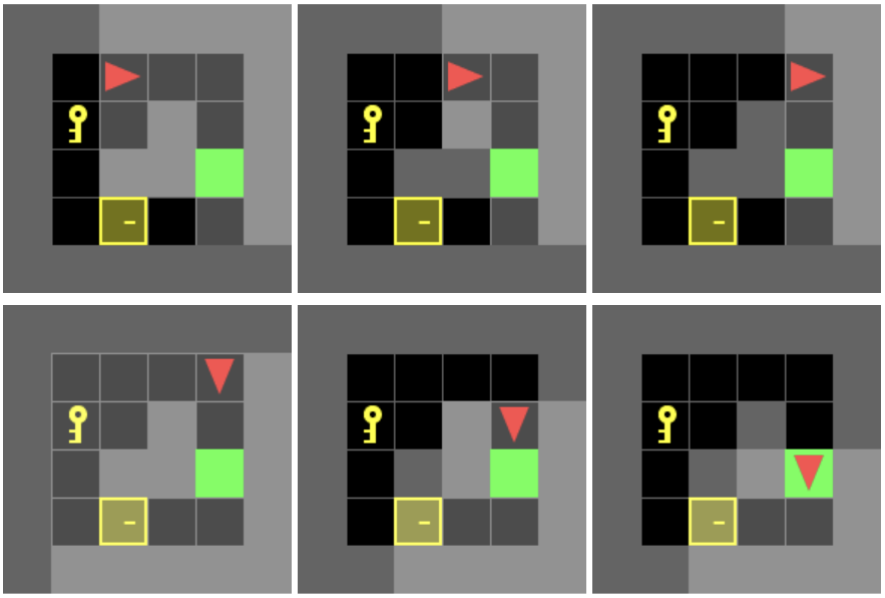

Fig. 2. Action sequences for the 6x6-direct.env map.

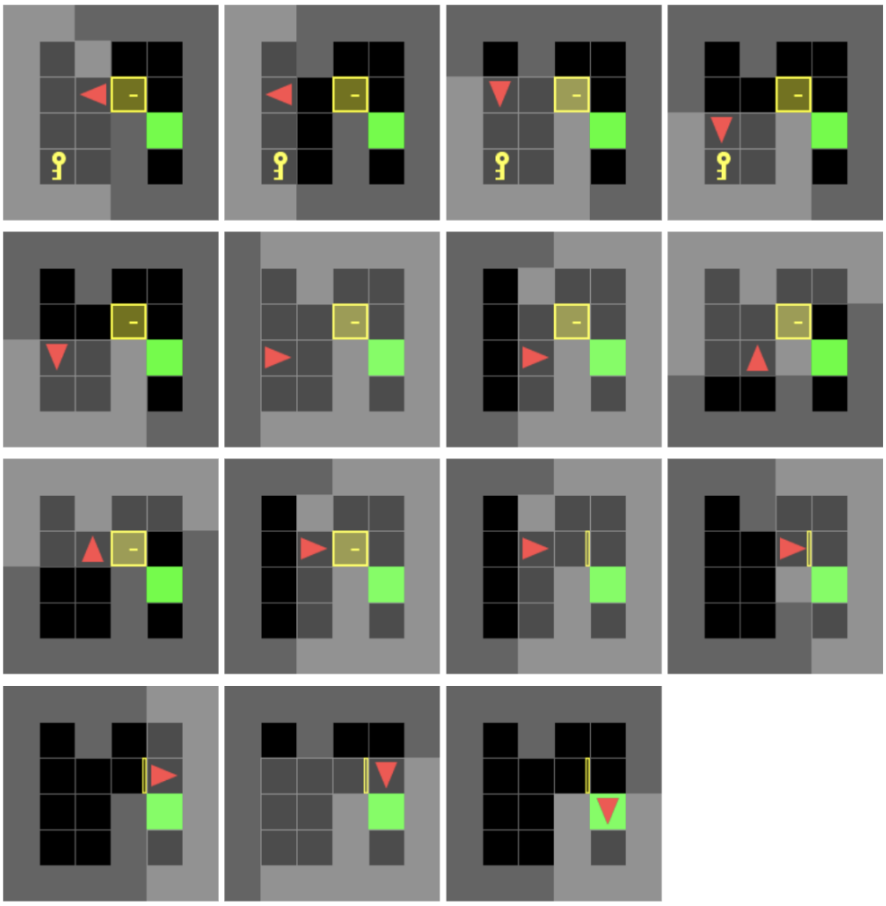

Fig. 3. Action sequences for the 6x6-normal.env map.

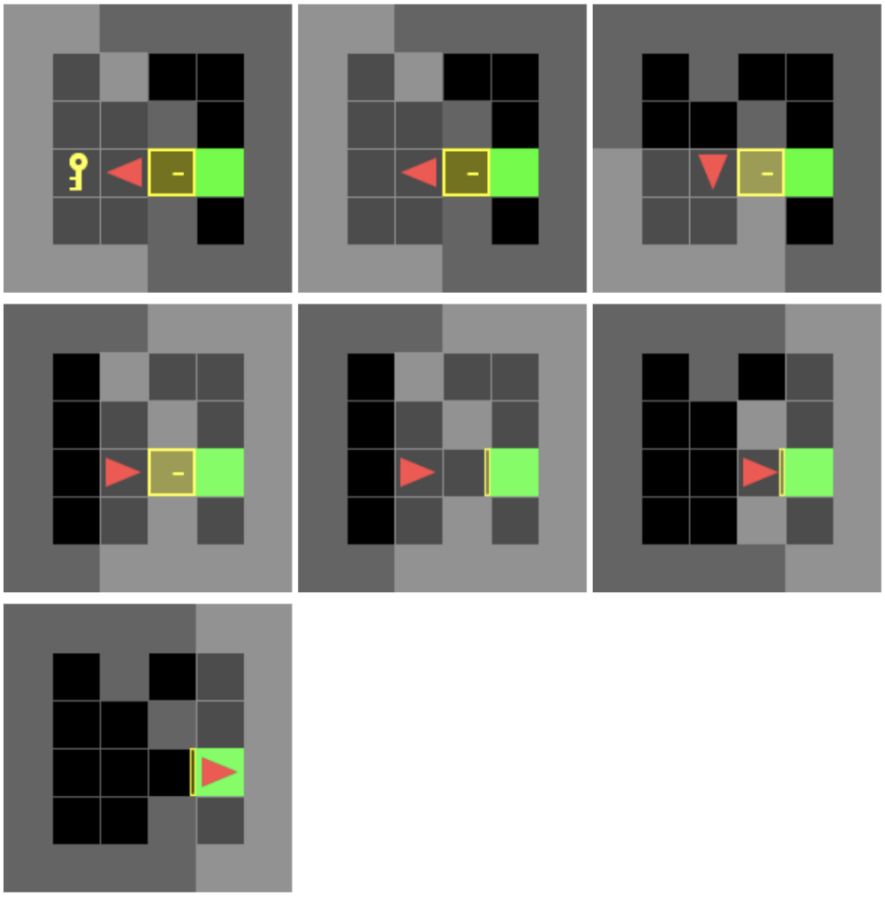

Fig. 4. Action sequences for the 6x6-shortcut.env map.

Map	Cost	Action Sequence
DoorKey-10x10-1.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-2.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-3.env	29	MF, TR, MF, MF, TL, MF, MF, MF, MF, MF, MF
DoorKey-10x10-4.env	50	MF, MF, MF, MF, MF, MF, TL, MF, PK, TL, MF, TL, MF, UD, MF, MF, TL, MF, MF
DoorKey-10x10-5.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF
DoorKey-10x10-6.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF
DoorKey-10x10-7.env	26	MF, TR, MF, MF, MF, TL, MF, MF, MF, MF
DoorKey-10x10-8.env	46	MF, MF, MF, MF, MF, MF, TL, MF, PK, TL, MF, TL, MF, UD, MF, MF, MF
DoorKey-10x10-9.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-10.env	32	MF, MF, MF, MF, MF, TR, MF, MF, TR, MF, MF, MF
DoorKey-10x10-11.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-12.env	53	MF, MF, MF, MF, MF, MF, TL, MF, PK, TL, MF, TL, MF, UD, MF, MF, TR, MF, MF, MF
DoorKey-10x10-13.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-14.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-15.env	29	MF, TR, MF, MF, TL, MF, MF, MF, MF, MF, MF

Map	Cost	Action Sequence
DoorKey-10x10-16.env	44	MF, MF, MF, MF, MF, TL, MF, PK, TL, TL, MF, UD, MF, MF, TL, MF, MF
DoorKey-10x10-17.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF
DoorKey-10x10-18.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF
DoorKey-10x10-19.env	26	MF, TR, MF, MF, MF, TL, MF, MF, MF, MF
DoorKey-10x10-20.env	40	MF, MF, MF, MF, MF, TL, MF, PK, TL, TL, MF, UD, MF, MF, MF
DoorKey-10x10-21.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-22.env	32	MF, MF, MF, MF, MF, TR, MF, MF, TR, MF, MF, MF
DoorKey-10x10-23.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-24.env	47	MF, MF, MF, MF, MF, TL, MF, PK, TL, TL, MF, UD, MF, MF, TR, MF, MF, MF
DoorKey-10x10-25.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-26.env	29	MF, MF, MF, MF, MF, TR, MF, MF, TL, MF, MF
DoorKey-10x10-27.env	29	MF, TR, MF, MF, TL, MF, MF, MF, MF, MF, MF
DoorKey-10x10-28.env	50	MF, MF, TL, MF, MF, PK, TR, MF, MF, MF, TR, MF, MF, UD, MF, MF, TL, MF, MF
DoorKey-10x10-29.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF
DoorKey-10x10-30.env	25	MF, MF, MF, MF, MF, TR, MF, MF, MF

Map	Cost	Action Sequence
DoorKey-10x10-31.env	26	MF, TR, MF, MF, MF, TL, MF, MF, MF, MF
DoorKey-10x10-32.env	46	MF, MF, TL, MF, MF, PK, TR, MF, MF, MF, TR, MF, MF, UD, MF, MF, MF
DoorKey-10x10-33.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-34.env	32	MF, MF, MF, MF, MF, TR, MF, MF, TR, MF, MF, MF
DoorKey-10x10-35.env	14	MF, TR, MF, MF, TL, MF
DoorKey-10x10-36.env	41	MF, MF, TL, MF, MF, PK, TL, MF, TL, MF, MF, UD, MF, MF, TL, MF

Fig. 5. Action sequences and costs for the random maps.

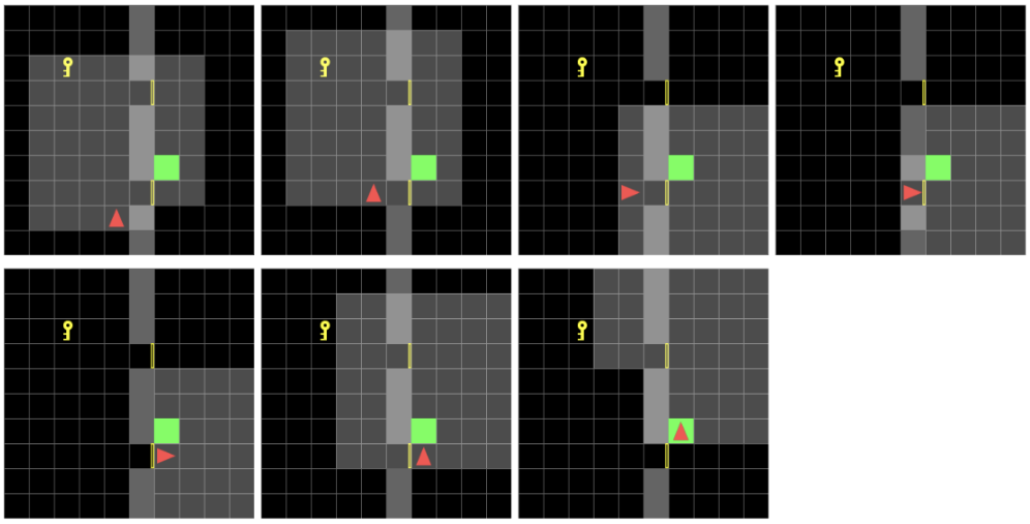

Fig. 6. Action sequences for the DoorKey-10x10-9.env map.

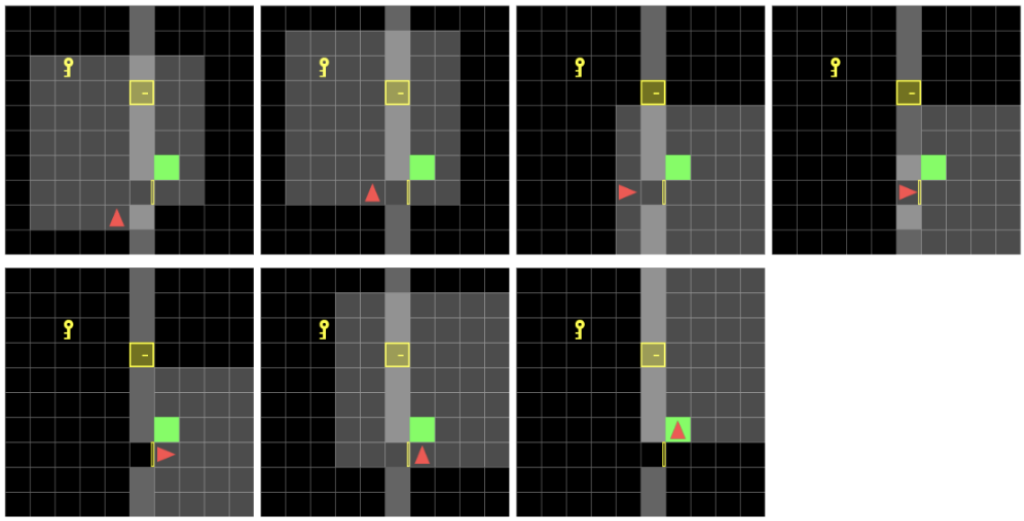

Fig. 7. Action sequences for the DoorKey-10x10-11.env map.

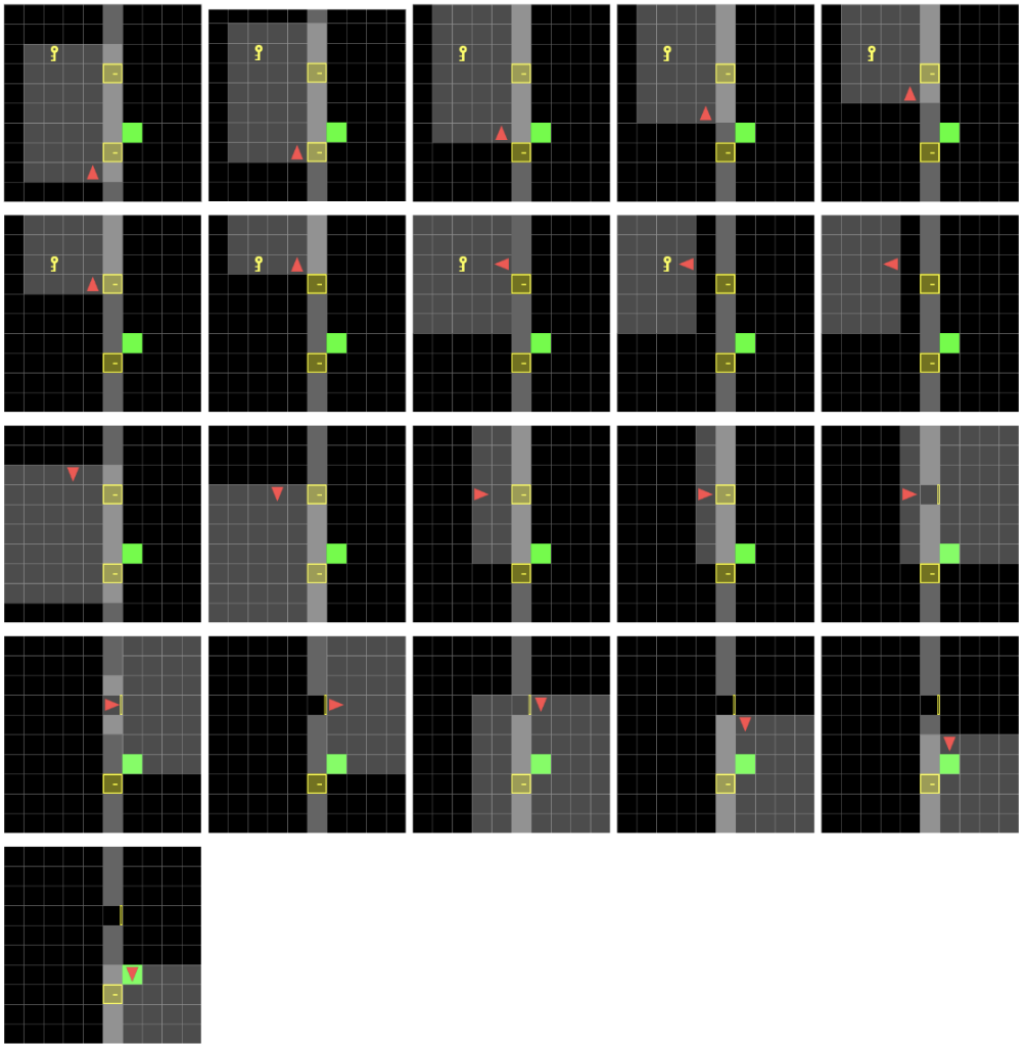

Fig. 8. Action sequences for the DoorKey-10x10-12.env map.